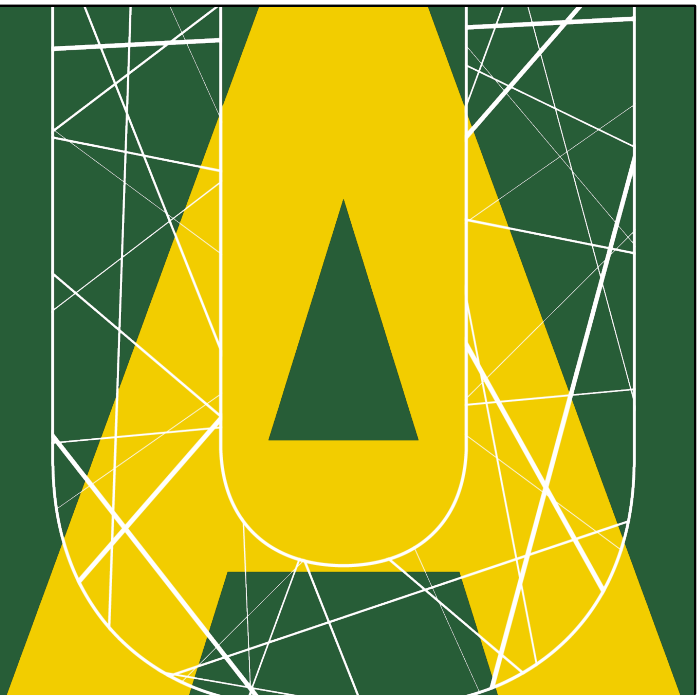
ECE 315

Steven Knudsen, PhD, PEng

Lecture 18 :

- Inter-Integrated Circuit (I²C)

UNIVERSITY OF ALBERTA



Last Lecture

- SPI and FreeRTOS
- RP2040 example
- Use Wokwi and make a custom chip, and analogue-to-digital converter (ADC)

This Lecture

- Inter-Integrated Circuit (I^2C)

Learning Objectives

- Explain the purpose and characteristics of the I²C communication protocol
- Differentiate I²C from other communication protocols such as SPI
- Describe the hardware and electrical characteristics of the I²C bus
  - Identify the roles of the SCL and SDA lines
  - Explain open-drain signaling and the need for pull-up resistors
- Illustrate the structure of an I²C communication transaction.
  - Recognize and describe START, STOP, ACK/NACK, address, and data byte sequences.
- Explain device addressing and bus topology.
  - Distinguish between 7-bit and 10-bit addressing formats controller/peripherals topology
- Discuss the concept of register-mapped peripherals
  - Understand how controllers read from and write to peripheral registers (e.g., MPU-6050 sensor).

What is I²C and Why Use it?

- The Inter-Integrated Circuit (I²C, or I2C, or rarely IIC) is a synchronous, multi-master/multi-slave, single-ended, serial communication bus invented in 1980 by Philips Semiconductors (now NXP Semiconductors)
- Similar to SPI, but only two wires and much lower data rates
- Originally designed for communication between ICs on the same board
- Widely used for sensors, EEPROMs, ADCs/DACs, RT clocks, display drivers
- I²C length is usually < 1 m; really only good for intra-board connections or short off-board distances

Wikipedia contributors. "I2C." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 24 Oct. 2025. Web. 18 Nov. 2025.

I²C vs SPI

Feature	I²C	SPI
Wires	2	4+
Speed	~100 kHz–3.4 MHz	Often tens of MHz
Multi-device support	Easy, shared bus	Requires chip select lines
Electrical	Open-drain + pull-ups	Push-pull
Complexity	Higher protocol complexity	Simple shift-register
Distance	< 1 m	Several meters (< 1 Mbps)
Full duplex?	No	Yes

I²C – good for simple sensors, low speed, up to 127 devices
SPI – good for high data rates, low latency, small number of devices

I²C Signaling

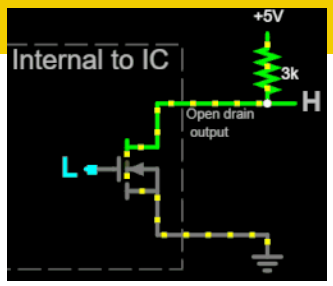
Two-wire Interface

- **SCL** – Serial Clock (driven by controller)
- **SDA** – Serial Data (bidirectional)

Electrical characteristics:

- Both lines are open-drain (MOSFETS) or open-collector (bipolar)
  - Cannot source current, only sink current to ground
- Require pull-up resistors
- Devices only pull the bus LOW; pull-ups bring lines HIGH

I²C Signaling



Two-wire Interface

- **SCL** – Serial Clock (driven by controller)
- **SDA** – Serial Data (bidirectional)

Electrical characteristics:

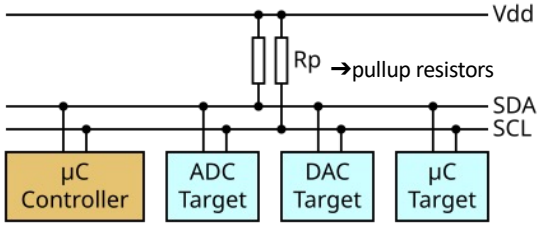
- Both lines are open-drain (MOSFETS) or open-collector (bipolar)
  - Cannot source current, only sink current to ground
- Require pull-up resistors
- Devices only pull the bus LOW; pull-ups bring lines HIGH

UNIVERSITY OF ALBERTA

Wikipedia contributors. "Open collector." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 12 Jun. 2025. Web. 18 Nov. 2025.

Topology

- Like SPI there is a Controller – Target/Peripheral architecture
- Controller – initiates all communication
- Target / Peripheral – responds to controller requests
- Multi-drop bus: many devices share the same two wires
  - Each device has a unique 7-bit or 10-bit address (7-bit most common)
  - Only the controller drives the clock
  - Multiple controllers possible (multi-master capable)
  - No chip select lines → scalable



https://commons.wikimedia.org/w/index.php?title=File:I2C_controller-target.svg&oldid=650413260

 Made with help from ChatGPT

Addressing – 7-bit

- Each device on the I2C bus has a unique address, either 7-bit format

Field:	S	I^2C address field							R/W'	A	I^2C message sequences...	P
Type	**Start**	**Byte 1**								**ACK**	**Byte X, etc. Rest of the read or write message goes here**	**Stop**
Bit position in byte X		**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**			
7-bit address pos		**7**	**6**	**5**	**4**	**3**	**2**	**1**				
Note		MSB						LSB	1 = Read 0 = Write			

UNIVERSITY OF ALBERTA

https://en.wikipedia.org/w/index.php?title=I2C&oldid=1318560729

Addressing – 10-bit

- Each device on the I2C bus has a unique address, either 10- bit format

Field:	S	10-bit mode indicator					Upper addr		R/W'	A	Lower address field								A	I^2C message sequences	P
Type		**Byte 1**									**Byte 2**										
Bit position in byte X		**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**		**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**		**Byte X etc.**	
Bit value		**1**	**1**	**1**	**1**	**0**	**X**	**X**	**X**		**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**		**Rest of the read or write**	
10-bit address pos	**Start**						**10**	**9**		**ACK**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**ACK**	**message goes here**	**Stop**
Note		Indicates 10-bit mode					MSB		1 = Read 0 = Write									LSB			

UNIVERSITY OF ALBERTA

https://en.wikipedia.org/w/index.php?title=I2C&oldid=1318560729

I²C Transaction

Field:	S	I²C address field							R/W'	A	I²C message sequences...	P
Type	Start	Byte 1								ACK	Byte X, etc. Rest of the read or write message goes here	Stop
Bit position in byte X		7	6	5	4	3	2	1	0			
7-bit address pos		7	6	5	4	3	2	1				
Note		MSB						LSB	1 = Read 0 = Write			

A complete message includes:

- START condition
- Address + R/W bit
- ACK/NACK from target
- One or more data bytes
- ACK/NACK after each data byte
- STOP condition
- START/STOP defined by SDA transitions while SCL is high.

I²C Transaction

Field:	S	I²C address field							R/W'	A	I²C message sequences...	P
Type	Start	Byte 1								ACK	Byte X, etc. Rest of the read or write message goes here	Stop
Bit position in byte X		7	6	5	4	3	2	1	0			
7-bit address pos		7	6	5	4	3	2	1				
Note		MSB						LSB	1 = Read 0 = Write			

A complete message includes:

- START condition
- Address + R/W bit
- ACK/NACK from target
- One or more data bytes
- ACK/NACK after each data byte
- STOP condition
- START/STOP defined by SDA transitions while SCL is high.

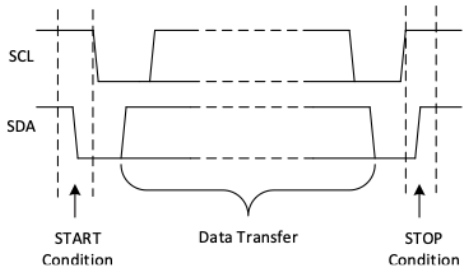


Figure 5. Example of START and STOP Condition

I²C Transaction

Field:	S	I²C address field							R/W'	A	I²C message sequences...	P
Type	Start	Byte 1								ACK	Byte X, etc. Rest of the read or write message goes here	Stop
Bit position in byte X		7	6	5	4	3	2	1	0			
7-bit address pos		7	6	5	4	3	2	1				
Note		MSB						LSB	1 = Read 0 = Write			

- The address byte has already been sent
- Each byte (address and data) is followed by an ACK from the receiver
- This message sequence is one byte long
  - A STOP follows…

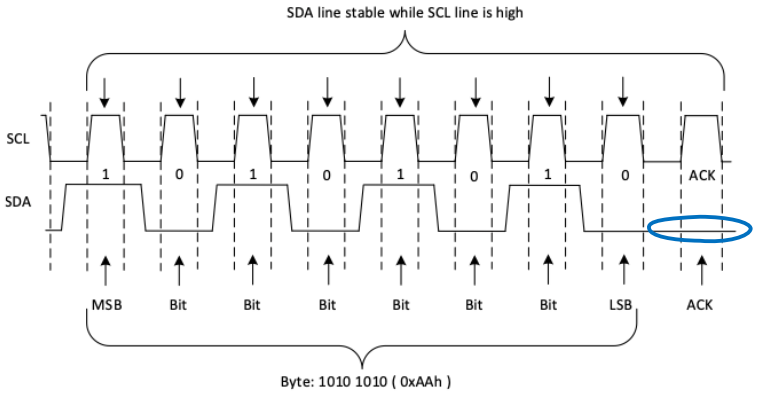


Figure 6. Example of Single Byte Data Transfer

Texas Instruments, Understanding the I2C Bus, SLVA704, June 2015

I²C Transaction

Field:	S	I²C address field							R/W'	A	I²C message sequences...	P
Type	Start	Byte 1								ACK	Byte X, etc. Rest of the read or write message goes here	Stop
Bit position in byte X		7	6	5	4	3	2	1	0			
7-bit address pos		7	6	5	4	3	2	1				
Note		MSB						LSB	1 = Read 0 = Write			

- Before the receiver can send an ACK, the transmitter must release the SDA line
- To send an ACK bit, the receiver pulls down the SDA line during the low phase of the ACK/NACK-related clock period, so that the SDA line is stable low during the high phase of the ACK/NACK-related clock period
- When the SDA line remains high during the ACK/NACK-related clock period, this is interpreted as a negative-ACK (NACK)
  - The receiver is unable to receive or transmit because it is performing some real-time function and is not ready to start communication with the controller
  - During the transfer, the receiver gets data or commands that it does not understand
  - During the transfer, the receiver cannot receive any more data bytes
  - A controller-receiver is done reading data and indicates this to the peripheral through a NACK

I²C Transaction - NACK

- Result for NACK depends on the context

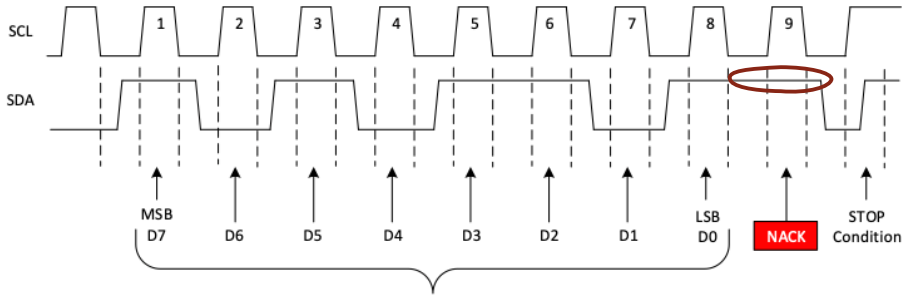


Figure 7. Example NACK Waveform

Texas Instruments, Understanding the I2C Bus, SLVA704, June 2015

Register-mapped Peripherals

- Very common for peripherals to be “register-mapped”
- For example, MPU-6050, an inexpensive and popular accelerometer-gyroscope is register-mapped

https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

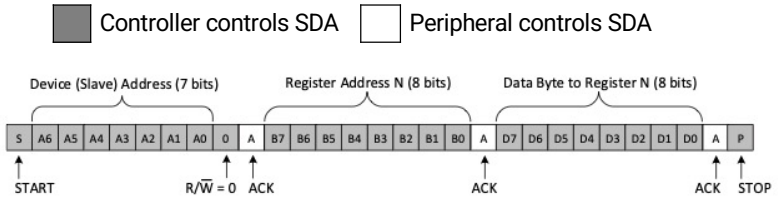
Register-mapped Peripherals

- Very common for peripherals to be “register-mapped”
- For example, MPU-6050, an inexpensive and popular accelerometer-gyroscope is register-mapped
- To get the X-axis acceleration need to read registers 0x3B and 0x3C

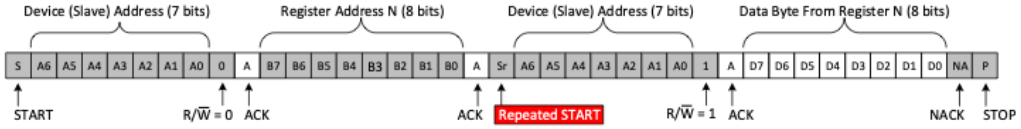
https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

Register-based Read/Write



Write to Peripheral Register



Read from Peripheral Register

UNIVERSITY OF ALBERTA

Texas Instruments, Understanding the I2C Bus, SLVA704, June 2015

19

Clock Stretching

- Targets may hold SCL low to delay controller
- Allows slow peripherals time to process data
- Supported by most controllers but often disabled by default
- Not present in SPI—unique feature of I²C

Summary

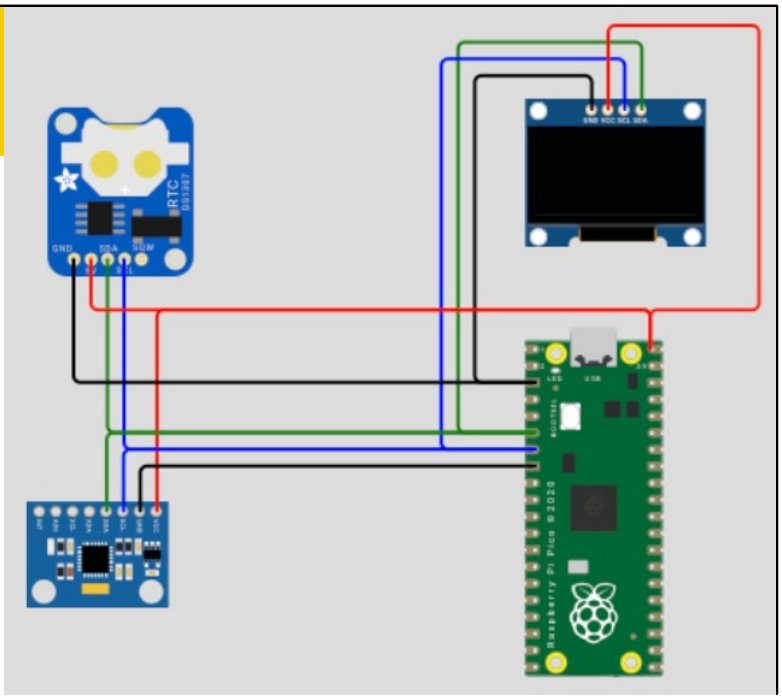
- I²C is a simple, scalable, two-wire digital interface
- Uses addressing rather than chip selects
- More complex protocol than SPI, but very flexible
- Supports multi-master and clock stretching
- Essential knowledge for embedded systems design

Wokwi Example – I²C Address Scanner

- A good first program to run with I²C peripherals is an address scanner
  - Tells you right away if your peripheral is connected and working
  - A good commissioning / debugging step
- On Wokwi, there is an ECE 315 Lecture 18 I2C Scanner example
  - Written in C
  - Scans all addresses except those reserved
    - https://wokwi.com/projects/447932239970111489

Peripherals

- Pico is connected to three I2C devices
  - DS1307 Real-time Clock
  - SSD1306 Mono-chrome 128x64 OLED
  - MPU-6050 3-axis accelerometer, 3-axis gyroscope



I2C Scanner Results

- Two addresses
  - 0x3C
  - 0x68
- Why not three addresses???

```
I2C Bus Scan
   0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
00 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
10 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
20 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
30 .  .  .  .  .  .  .  .  .  .  .  .  @  .  .  .
40 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
50 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
60 .  .  .  .  .  .  .  .  @  .  .  .  .  .  .  .
70 .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
Done.
```

UNIVERSITY OF ALBERTA

Next Lecture

- Parallel Interfacing
- Busses

Questions

UNIVERSITY OF ALBERTA